Guía n°

Hello World con RabbitMQ en Docker

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Subdirección de Diseño
Curricular e Instruccional
202

Guía n°

Hello World con RabbitMQ en Docker

Sigla	Asignatura	Experiencia de Aprendizaje
Tiempo	**Modalidad de Trabajo**	**Indicadores de Logro**

Contexto

- Este es un tutorial paso-a-paso para crear tu primer sistema de mensajería con RabbitMQ. Aprenderás a:
- ✅ Levantar RabbitMQ con Docker
- ✅ Crear una cola de mensajes
- ✅ Implementar un productor (envía mensajes)
- ✅ Implementar un consumidor (recibe mensajes)
- ✅ Probar la comunicación end-to-end

Documentación Oficial Base: - Tutorial Oficial RabbitMQ Hello

https://www.rabbitmq.com/tutorials/tutorial-one-spring-amqp

RabbitMQ Java Client Library
https://www.rabbitmq.com/client-libraries/java-client
Java Client API Guide
https://www.rabbitmq.com/client-libraries/java-api-guide
Spring AMQP Documentation
https://docs.spring.io/spring-amqp/reference/
RabbitMQ AMQP 0-9-1 Protocol

https://www.rabbitmq.com/protocol.html
RabbitMQ Docker Official Images
https://hub.docker.com/_/rabbitmq

Requisitos Previos

Antes de comenzar, asegúrate de tener instalados:

Docker Instalado

Verifica que Docker está instalado:

```
docker --version
 # Debe mostrar algo como: Docker version XXX Build XXX
```

Si no tienes Docker, Revisa los Videos en tu AVA de como instalar Docker Desktop

Java JDK 8+ (recomendado JDK 11 o superior)**)**

Verifica que tienes Java:

```
java --version

# Debe mostrar: openjdk version "11" o superior
```

Comenzamos

Paso 1: Levantar RabbitMQ con Docker

Tienes dos opciones: usar Docker directamente o Docker Compose. Recomendamos Docker Compose por facilidad.

Crea un archivo docker-compose.yml en tu carpeta del proyecto:

```yaml
version: '3.8'

services:
  rabbitmq:
    image: rabbitmq:4.2-management
    container_name: rabbitmq
    ports:
      - "5672:5672"    # AMQP Protocol Port
      - "15672:15672"  # Management UI Port
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
    volumes:
      - rabbitmq_data:/var/lib/rabbitmq
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
```

```
volumes:
  rabbitmq_data:
```

Luego levanta el servicio:

```
docker-compose up -d
```

Verifica que está corriendo:

```
docker-compose ps
```

Verificar que RabbitMQ está Activo

Puedes verificar de varias formas:

Método 1: Ver logs

```
docker logs rabbitmq
 # Debe mostrar líneas como "RabbitMQ started and ready to accept connections"
```

Método 2: Acceder a la UI web

Abre en tu navegador: **http://localhost:15672**

- Usuario: guest
- Contraseña: guest

Deberías ver un dashboard con información de RabbitMQ.

Método 3: Verificar con rabbitmqctl

```
docker exec rabbitmq rabbitmq-diagnostics ping
 # Debe mostrar: ok
```

Si todo está bien, continúa al siguiente paso.

Paso 2: Crear el Proyecto Spring Boot

2.1 Opción A: Usar Spring Initializr (Recomendado)

Visita: https://start.spring.io

Configura así:

- Project: Maven Project
- Language: Java
- Spring Boot: 3.1.x (o superior)
- Group: com.example
- Artifact: rabbitmq-tutorials
- Packaging: Jar
- Java: 11 (o tu versión)
- Dependencies:
  - Spring AMQP
  - Spring Boot Web (opcional, para REST endpoints)

Descarga, descomprime e importa en tu IDE.

2.2 Opción B: Crear manualmente con Maven

Bash

```
mvn archetype:generate \
 -DgroupId=com.example \
 -DartifactId=rabbitmq-tutorials \
 -DarchetypeArtifactId=maven-archetype-quickstart \
 -DinteractiveMode=false
```

cd rabbitmq-tutorials

2.3 Agregar dependencia de Spring AMQP al pom.xml

Abre pom.xml y añade esta dependencia en la sección <dependencies>:

xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Referencia oficial:

- https://docs.spring.io/spring-amqp/reference/

Paso 3: Configurar Spring Boot para RabbitMQ

3.1 Crear application.yml

Crea o modifica el archivo src/main/resources/application.yml:

```
spring:
 application:
  name: rabbitmq-tutorials

 rabbitmq:
  host: localhost
  port: 5672
  username: guest
  password: guest
  virtual-host: /
```

```
  amqp:
    template:
      retry:
        enabled: true
        initial-interval: 1000
        max-interval: 30000
        multiplier: 2.0

server:
  port: 8080
```

Explicación:

- host: localhost - RabbitMQ está en tu máquina local
- port: 5672 - Puerto AMQP estándar
- username/password - Credenciales por defecto
- virtual-host: / - Virtual host por defecto

Referencia oficial:

- https://docs.spring.io/spring-boot/reference/application-properties.html

Paso 4: Crear la Configuración de RabbitMQ

Crea la clase src/main/java/com/example/RabbitMQConfig.java:

java

```
package com.example;

import org.springframework.amqp.core.Queue;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

/**
 * Configuración de RabbitMQ
```

```
 *
 * Basada en:
 * https://www.rabbitmq.com/tutorials/tutorial-one-spring-amqp
 * https://docs.spring.io/spring-amqp/reference/
 */
@Configuration
public class RabbitMQConfig {

    /**
     * Define la cola "hello"
     *
     * Esta cola es idempotente:
     * - Si no existe, la crea
     * - Si ya existe, la reutiliza
     *
     * durable=false: se borra si RabbitMQ se reinicia
     * (En producción, normalmente usarías durable=true)
     */
    @Bean
    public Queue helloQueue() {
        return new Queue("hello", false);
    }
}
```

Explicación:

- @Configuration - Marca la clase como configuración de Spring
- @Bean - Registra la cola como bean de Spring
- Queue("hello", false) - Crea una cola llamada "hello", no durable
- El método retorna un bean Queue que Spring AMQP gestiona automáticamente

Referencia oficial:

- https://www.rabbitmq.com/tutorials/tutorial-one-spring-amqp

Paso 5: Crear el Productor (Sender)

Crea la clase src/main/java/com/example/Sender.java:

java

```
package com.example;

import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 * Productor (Sender) - Envía mensajes a la cola 'hello'
 *
 * Basado en:
 * https://www.rabbitmq.com/tutorials/tutorial-one-spring-amqp
 * https://docs.spring.io/spring-amqp/reference/
 */
@Component
public class Sender {

    /**
     * RabbitTemplate es la clase principal de Spring AMQP
     * para enviar mensajes.
     *
     * Proporciona métodos simplificados para:
     * - convertAndSend(): convierte objeto a JSON y envía
     * - convertSendAndReceive(): espera respuesta
     *
  Referencia:

  https://docs.spring.io/spring-amqp/docs/current/api/org/springframework/amqp/rabbit/core/RabbitTemplate.html


    @Autowired
```

```
    private RabbitTemplate rabbitTemplate;

    /**
     * Método para enviar un mensaje simple
     *
     * Parámetros:
     * - exchange: "" (vacío) significa usar el exchange por defecto
     * - routingKey: "hello" (nombre de la cola destino)
     * - message: el contenido del mensaje
     */
    public void sendMessage(String message) {
        try {
            String timestamp = LocalDateTime.now()
                .format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));

            String fullMessage = String.format(
                "[%s] %s",
                timestamp,
                message
            );

            // Enviar el mensaje a la cola
            // exchange="" usa el exchange por defecto (DIRECT)
            // routingKey="hello" especifica la cola destino
            rabbitTemplate.convertAndSend("hello", fullMessage);

            System.out.println("[✓] Mensaje enviado: '" + fullMessage +
"'");

        } catch (Exception e) {
            System.err.println("[✗] Error enviando mensaje: " +
e.getMessage());
            e.printStackTrace();
        }
    }
```

```
    /**
     * Versión sobrecargada con exchange explícito
     * (para casos más avanzados)
     */
    public void sendMessage(String exchange, String routingKey,
String message) {
        try {
            rabbitTemplate.convertAndSend(exchange, routingKey,
message);
            System.out.println("[✓] Mensaje enviado a exchange='" +
exchange
                    + "', routingKey='" + routingKey + "': '" + message + "'");
        } catch (Exception e) {
            System.err.println("[✗] Error enviando mensaje: " +
e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Explicación del código:

Concepto	Explicación
RabbitTemplate	Clase principal de Spring AMQP para enviar mensajes. Simplifica la API de bajo nivel
@Autowired	Spring inyecta automáticamente la instancia de RabbitTemplate
convertAndSend()	Convierte el objeto Java a JSON y lo envía a RabbitMQ
exchange=""	Usa el exchange por defecto (no especificamos exchange)
routingKey="hello"	Especifica a qué cola va el mensaje

Referencia oficial:

- https://docs.spring.io/spring-amqp/docs/current/api/org/springframework/amqp/rabbit/core/RabbitTemplate.html

Paso 6: Crear el Consumidor (Receiver)

Crea la clase src/main/java/com/example/Receiver.java:

java

```
package com.example;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 * Consumidor (Receiver) - Recibe mensajes de la cola 'hello'
 *
 * Basado en:
 * https://www.rabbitmq.com/tutorials/tutorial-one-spring-amqp
 * https://docs.spring.io/spring-amqp/reference/
 */
@Component
public class Receiver {

  /**
   * Este método se ejecuta automáticamente cada vez que
   * llega un mensaje a la cola 'hello'
   *
   * La anotación @RabbitListener:
   * - Registra este método como listener de mensajes
   * - queues="hello" especifica la cola a escuchar
   * - Spring AMQP maneja automáticamente:
   *   - Conexión a RabbitMQ
   *   - Deserialización del mensaje
   *   - Confirmación (ack) del mensaje
```

```
    *   - Manejo de excepciones
    *
    * Referencia:
    *
https://docs.spring.io/spring-amqp/docs/current/api/org/springframe
work/amqp/rabbit/annotation/RabbitListener.html
    */
    @RabbitListener(queues = "hello")
    public void receiveMessage(String message) {
        try {
            String timestamp = LocalDateTime.now()
                .format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));

            System.out.println(
                "[" + timestamp + "] [✓] Mensaje recibido: '" + message + "'"
            );

            // Aquí va tu lógica de procesamiento del mensaje
            // Si lanzas una excepción, Spring AMQP reintentará el mensaje
            // según la configuración de retry

        } catch (Exception e) {
            System.err.println("[✗] Error procesando mensaje: " +
e.getMessage());
            e.printStackTrace();
            // Relanzar la excepción para que Spring AMQP maneje el retry
            throw new RuntimeException(e);
        }
    }

    /**
    * Método alternativo que recibe el mensaje como un objeto más
complejo
    * (Este es un ejemplo avanzado)
    */
    @RabbitListener(queues = "hello")
```

```
   public void receiveMessageAdvanced(String message,
                        org.springframework.amqp.core.Message
rawMessage,
                        org.springframework.amqp.rabbit.core.Channel
channel)
         throws Exception {
      try {
         System.out.println("[✓] Mensaje recibido: '" + message + "'");
         System.out.println("  - Content-Type: " +
rawMessage.getMessageProperties().getContentType());
         System.out.println("  - Timestamp: " +
rawMessage.getMessageProperties().getTimestamp());

         // Confirmación manual del mensaje (si usas manual acks)
         //
channel.basicAck(rawMessage.getMessageProperties().getDeliveryTag
(), false);

      } catch (Exception e) {
         System.err.println("[✗] Error: " + e.getMessage());
         // Rechazar el mensaje y requearlo (si usas manual acks)
         //
channel.basicNack(rawMessage.getMessageProperties().getDeliveryTa
g(), false, true);
         throw e;
      }
   }
}
```

Explicación del código:

Concepto	Explicación
@RabbitListener	Anotación que registra el método como listener de mensajes
queues="hello"	Especifica que este listener escucha la cola 'hello'

String message	Spring AMQP deserializa automáticamente el mensaje
Channel	Acceso al canal AMQP para confirmación manual (manual acks)
basicAck()	Confirma que el mensaje fue procesado exitosamente
basicNack()	Rechaza el mensaje y lo devuelve a la cola

Referencia oficial:

- https://docs.spring.io/spring-amqp/docs/current/api/org/springframework/amqp/rabbit/annotation/RabbitListener.html

Paso 7: Crear la Aplicación Principal

Crea o modifica src/main/java/com/example/Application.java:

java

```
package com.example;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import java.util.Scanner;

/**
 * Aplicación principal de RabbitMQ con Spring Boot
 *
 * Basada en:
 * https://www.rabbitmq.com/tutorials/tutorial-one-spring-amqp
 * https://docs.spring.io/spring-boot/reference/
 */
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        // Levanta la aplicación Spring Boot
        ApplicationContext ctx = SpringApplication.run(Application.class,
```

```
args);

        // Obtén el bean Sender
        Sender sender = ctx.getBean(Sender.class);

        System.out.println("\n" + "=".repeat(60));
        System.out.println("RabbitMQ - Hello World con Spring Boot");
        System.out.println("=".repeat(60));
        System.out.println("[✓] Aplicación iniciada correctamente");
        System.out.println("[✓] RabbitMQ conectado en localhost:5672");
        System.out.println("[✓] Cola 'hello' lista para usar\n");

        // Menú interactivo
        Scanner scanner = new Scanner(System.in);
        boolean running = true;

        while (running) {
            System.out.println("\n--- Menú ---");
            System.out.println("1. Enviar mensaje");
            System.out.println("2. Enviar múltiples mensajes");
            System.out.println("3. Salir");
            System.out.print("Selecciona opción (1-3): ");

            String option = scanner.nextLine().trim();

            switch(option) {
                case "1":
                    System.out.print("Escribe el mensaje: ");
                    String message = scanner.nextLine();
                    sender.sendMessage(message);
                    break;

                case "2":
                    System.out.print("¿Cuántos mensajes? ");
                    try {
```

```
                int count = Integer.parseInt(scanner.nextLine());
                for (int i = 1; i <= count; i++) {
                  sender.sendMessage("Mensaje #" + i + " - Hello
RabbitMQ!");
                  Thread.sleep(500); // Pequeña pausa
                }
              } catch (NumberFormatException e) {
                System.out.println("[✗] Número inválido");
              } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
              }
              break;

            case "3":
              System.out.println("[✓] Saliendo...");
              running = false;
              break;

            default:
              System.out.println("[✗] Opción no válida");
          }
        }

        scanner.close();
        System.exit(0);
    }
}
```

Explicación:

- @SpringBootApplication - Marca la clase como aplicación Spring Boot
- SpringApplication.run() - Levanta la aplicación
- ctx.getBean(Sender.class) - Obtiene el bean Sender para enviar mensajes

- El programa proporciona un menú interactivo para enviar mensajes

Referencia oficial:

- https://docs.spring.io/spring-boot/reference/features/spring-application.html

Paso 8: Construir y Ejecutar

8.1 Compilar el proyecto

bash
Compilar con Maven
mvn clean package

Debe generar: target/rabbitmq-tutorials-0.0.1-SNAPSHOT.jar

8.2 Ejecutar la aplicación

bash
Opción 1: Con Maven
mvn spring-boot:run

Opción 2: Ejecutar el JAR directamente
java -jar target/rabbitmq-tutorials-0.0.1-SNAPSHOT.jar

Deberías ver:

[✓] Aplicación iniciada correctamente
[✓] RabbitMQ conectado en localhost:5672
[✓] Cola 'hello' lista para usar

--- Menú ---
1. Enviar mensaje

2. Enviar múltiples mensajes
3. Salir
Selecciona opción (1-3):

8.3 Probar envío y recepción de mensajes

En el menú, selecciona opción 1 y escribe un mensaje:

Selecciona opción (1-3): 1
Escribe el mensaje: Hola RabbitMQ desde Spring Boot!
[✓] Mensaje enviado: '[14:30:45.123] Hola RabbitMQ desde Spring Boot!'
[14:30:45.234] [✓] Mensaje recibido: '[14:30:45.123] Hola RabbitMQ desde Spring Boot!'

¡Éxito! El mensaje pasó del Sender a través de RabbitMQ al Receiver.

Paso 9: Crear un Endpoint REST (Opcional)

Si quieres una forma más profesional de enviar mensajes, crea un REST controller:

Crea src/main/java/com/example/MessageController.java:

java

```
package com.example;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

/**
 * REST Controller para enviar mensajes a RabbitMQ
```

```
 *
 * Basado en:
 *
https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html
 */
@RestController
@RequestMapping("/api/messages")
public class MessageController {

    @Autowired
    private Sender sender;

    /**
     * Enviar un mensaje via POST
     *
     * Ejemplo:
     * curl -X POST http://localhost:8080/api/messages |
     *   -H "Content-Type: application/json" |
     *   -d '{"message":"Hello from REST API!"}'
     */
    @PostMapping
    public ResponseEntity<String> sendMessage(@RequestBody
MessageRequest request) {
        try {
            sender.sendMessage(request.getMessage());
            return ResponseEntity.ok("Mensaje enviado: " +
request.getMessage());
        } catch (Exception e) {
            return ResponseEntity.badRequest().body("Error: " +
e.getMessage());
        }
    }

    /**
     * Enviar un mensaje via GET (más simple, menos recomendado)
```

```
    *
    * Ejemplo:
    * curl "http://localhost:8080/api/messages/send?
message=Hello%20World"
    */
   @GetMapping("/send")
   public ResponseEntity<String> sendMessageGet(
         @RequestParam(name = "message") String message) {
      try {
         sender.sendMessage(message);
         return ResponseEntity.ok("Mensaje enviado: " + message);
      } catch (Exception e) {
         return ResponseEntity.badRequest().body("Error: " +
e.getMessage());
      }
   }

   /**
    * Clase DTO para recibir el mensaje en JSON
    */
   public static class MessageRequest {
      private String message;

      public MessageRequest() {}

      public MessageRequest(String message) {
         this.message = message;
      }

      public String getMessage() {
         return message;
      }

      public void setMessage(String message) {
         this.message = message;
      }
```

```
  }
}
```

Luego puedes enviar mensajes con curl:

bash

```
# Opción 1: POST con JSON
curl -X POST http://localhost:8080/api/messages \
  -H "Content-Type: application/json" \
  -d '{"message":"Hola desde REST API!"}'

# Opción 2: GET con parámetro
curl "http://localhost:8080/api/messages/send?message=HelloWorld"
```

Referencia oficial:

- https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html

Paso 10: Monitorear con el Dashboard de RabbitMQ

Abre: http://localhost:15672

- Usuario: guest
- Contraseña: guest

Secciones útiles:

Queues and Streams

- Ver la cola "hello"
- Número de mensajes en la cola
- Número de consumidores conectados
- Opciones para purgar la cola

Connections

- Conexiones activas de productor/consumidor

- Información del cliente (dirección IP, puerto)

Channels

- Canales abiertos
- Estado de cada canal

Overview

- Estadísticas generales
- Memory usage
- Número total de mensajes publicados

Solución de Problemas

Problema: "Connection refused on localhost:5672"

Causas posibles:

1. RabbitMQ no está levantado
2. RabbitMQ está en otro puerto
3. RabbitMQ se crashed

Soluciones:

bash

```
# Verificar que RabbitMQ está corriendo
docker ps | grep rabbitmq

# Si no está, levantarlo
docker-compose up -d

# Ver logs de RabbitMQ
docker logs rabbitmq

# Esperar a que esté completamente listo (30 segundos)
docker-compose ps
# HEALTH debe ser "healthy"
```

Problema: "Unknown host exception"

Asegúrate que application.yml tiene la configuración correcta:

```
spring:
rabbitmq:
host: localhost
port: 5672
```

Problema: "Spring Boot no conecta a RabbitMQ"

Verifica que:

1. RabbitMQ está levantado: docker ps | grep rabbitmq
2. El puerto 5672 está abierto: netstat -an | grep 5672
3. Las credenciales en application.yml son correctas: guest/guest
4. El virtual-host / existe (es el por defecto)

Problema: No se reciben mensajes

Causa común: El Receiver no está levantado

Solución:

1. Asegúrate de compilar con mvn clean package
2. Ejecutar con java -jar target/...jar
3. Esperar a que veas "RabbitMQ conectado"
4. Luego enviar mensajes desde otra terminal

Comparación: Python vs Spring Boot

Aspecto	Python (Pika)	Spring Boot (Spring AMQP)
Librería	pika	spring-boot-starter-amqp
Conexión	Explícita (BlockingConnection)	Automática (RabbitTemplate)
Declaración Cola	Manual (queue_declare)	@Bean o @Configuration

Envío	channel.basic_publish()	rabbitTemplate.convertAndSend()
Recepción	callback + start_consuming()	@RabbitListener
Manejo Errores	Try/catch manual	Framework automático
Complexity	Media (más verboso)	Baja (más declarativo)
Production Ready	Sí (con ajustes)	Muy preparado

Configuración de Producción (Recomendaciones)

Para un entorno de producción, considera estos ajustes en application.yml:

text

```
spring:
 rabbitmq:
   # Conexión
   host: rabbitmq.production.com
   port: 5672
   username: ${RABBITMQ_USER}
   password: ${RABBITMQ_PASSWORD}
   virtual-host: /

   # Pool de conexiones
   connection-timeout: 10s

   # Auto-recovery
   requested-heartbeat: 30

   # Retry
   template:
    retry:
```

```
    enabled: true
    initial-interval: 1000
    max-interval: 30000
    multiplier: 2.0
    max-attempts: 3

# Listener
amqp:
 listener:
  type: simple
  simple:
   prefetch: 1       # Procesar 1 mensaje a la vez
   concurrency: 1-3    # 1-3 threads de consumo
   acknowledge-mode: manual  # Confirmación manual
   max-attempts: 3     # Reintentos
   default-requeue-rejected: false
```

Referencia oficial:

- https://docs.spring.io/spring-boot/reference/application-properties.html